

AM50 Topic 4: Information Theory

Applying Shannon's Entropy to Word Frequency in Text

Today's computer lab has three aims:

1. Learn about how to count the frequency distributions of words in a passage of text.
2. Compare the frequency distributions in a few (famous and not so famous) texts, and compare your measurements to Zipf's law.
3. Use this information to measure *Shannon's entropy* for these texts.

1 Shannon's Entropy, and the Main point of the Lab

Our paper this week is Shannon's Mathematical Theory of Information, in which Shannon discovers how to measure the information content of, e.g. an English text. At the heart of Shannon's idea is that we should think of a text as emanating from an information source, which *probabilistically* emits letters or words at some predetermined frequencies.

Namely, when writing the term paper for your Literature class, Shannon envisions you as having some set of words at your disposal, and your brain chooses words from this distribution and puts them on the page. To the extent that you are choosing your words in a not-random fashion, the particular order of the words in your brain is the information content of the document.

Shannon measures the information through the *entropy* of the text. Suppose that the your brain contains M different words, and that the i^{th} word has a frequency f_i . Then the entropy is given by

$$H = - \sum_{i=1}^M f_i \log_2(f_i). \quad (1)$$

Note that the logarithm here is base 2.¹

As Shannon argues, if you then consider writing a document of N words, the number of possible combinations of words you can choose is

$$\# \text{ of Words} = 2^{HN}.$$

Thus the particular combination of words that you have chosen for your essay is one out of this many possibilities. Note that the higher the entropy, H , the more possible documents your brain can put together—and the more unique each particular one is! Shannon defines

¹ This, and today's entire lab, is a simplification of the real calculation. In reality, the statistical structure of a language does not just depend on the word frequencies but on the relationship of different words to each other. So for example, a sentence beginning with the word "So" is more likely to have a second word "for" than "example". We are neglecting this effect in today's lab. If you are interested you could turn a more precise analysis into a final project!"

H with respect to the base 2 logarithm because this way you can imagine coding all of the words in a binary counter—i.e. with a series of 0's and 1's, like a computer. *Then the entropy H measures the number of binary digits required to code a word of the language.*

So the question is: What is H ? What is your personal H ? What is the H for the English language? Are they all the same? Or do better (or worse) writers have a higher value for H ?

The goal of today's lab is to make some simple measurements to begin to answer these questions.

2 Counting Word Frequency Distributions

Our goal is to give you the needed tools to compute the entropy of a text. Once you can do this you can measure the entropy of a few representative texts that we are going to give you. In your homework you will measure the entropy of your own writing.

To count the word frequencies, you should use the Python program `wordcount.py`, supplied to you. You probably don't *need* to modify this file but it can definitely be improved using some techniques from lecture. This program takes a filename as input and then outputs the word frequency distribution.

The program contains several advanced concepts that you should be aware of but it is not necessary that you understand all the details. However we will illustrate how a few of the commands work so that you get the idea.

1. First it opens the file. The file **must** be a '.txt' file—it cannot read in Microsoft Word files for example. You can easily make a .txt file by copying the bit of text, and then saving it as a .txt file within Microsoft Word. Better yet, use Notepad (Windows) or TextEdit (Mac). On the course web site we have given you a txt file of the Gettysburg Address, called "gettysburg.txt".

```
f = open(filename, 'r')
s = f.read()
f.close()
```

This command opens the file and Python assigns the file a file identifier (or handle) `f`. Then we "read" the text into a long string `s`. Since we are now done with the file, it's a good idea to close it.

2. Now the program converts the file into a Python data structure called a *dictionary*. First, the long string is "split" into a list of individual words called `words`. We use "regular expressions" to split the long string on punctuation marks and spaces. This is the hard part.

```
words = re.split("[\W\s\!+\.\.,+\?+\"]", s)
```

This requires the `re` module so we need `import re` at the top.

Now we turn the list into a dictionary called `dicto` using

```
dicto = {}
for word in words:
    if word in dicto.keys():
        dicto[word] += 1
    else:
        dicto[word] = 1
```

The dictionary holds words as “keys” and the number of times they appear in the text as “values”. Type `print dicto` to look at it.

3. Afterwards you can check the size of dictionary (number of keys), i.e. the number of *distinct* words, by `len(dicto)`.
4. The function returns the dictionary and an array of words and their relative frequency in the original text (as a percentage).

```
return dicto, np.array(sorted(dicto.values())[:-1]) / float(len(
    words)-empties) * 100
```

To use `wordcount.py` in a *separate* Python file in the same directory, type

```
import wordcount as wc
```

and then read the text file into Python by typing

```
results0, freq0 = wc.wordcount('gettysburg.txt')
```

Exercise 1

Use `wordcount.py` to compute the frequency distribution of words in the Gettysburg address. Plot the distribution on a log-log plot by typing

```
p0, = plt.loglog(freq0, 'g.')
```

The ‘g.’ plots green points. Remember to first use `import matplotlib.pyplot as plt`. Add axes labels and a legend.

We now want you to take two other documents and again measure the word frequency distribution, to see how it changes from document to document.

Exercise 2

We have put two text files of news articles on the course web site, both of which discuss the non-government shut down that happened a few years ago. The first article is from the Huffington post, and the second from Fox News. They have completely different slants as you would expect. Measure the word frequency distribution of this and plot it on top of the plot you made for the Gettysburg address in but in a different color or shape. Add axes labels and a legend.

Exercise 3

Now make a text file of some article or document of interest to you, that you can find on the web. Make your document into a .txt file. Measure the word frequency distribution of the document and plot it on your plot.

You should see that the word frequency distributions of your three documents have a surprising similarity. Compare the top few ranked words in the documents—are they the same or different? Look further down the list, when does the difference arise?

There is a famous law that governs the frequency distribution of words—called Zipf’s law. See

http://en.wikipedia.org/wiki/Zipf%27s_law.

This says that the frequency distribution of words in a document obeys

$$f_k = \frac{C}{k},$$

where f_k is the frequency of occurrence of the k^{th} word and C is a constant.

Exercise 4

Does your data agree with Zipf’s law? Why or why not?

3 Calculating the Entropy

Finally, Shannon defined the entropy according to Equation (1).

Exercise 5

Write a function to compute the entropy of a set of word frequencies. The function should take in the list “freq” that is output by the `wordcounts.py` program, and compute the entropy given in Equation (1). Make sure when you compute your entropy you use the base 2 logarithm, given by the Python command `log2`.

Exercise 6

Use your function to measure the entropy of each of the texts. Is there more information content in either the Fox news article or the Huffington post article?